

Finals Project Architecture Report

QDate — As-Built Implementation

322535436 | 313308504 | 207801796

Abstract

This project is QDate, an innovative dating application designed to address the "paradox of choice" in modern digital dating. While traditional platforms maximize swipes and matches, QDate focuses on meaningful decision-making through enforced artificial scarcity. The system operates in two distinct phases: a 14-day learning period with daily matches to gather behavioral data, followed by a curated weekly match phase.

By using machine learning to analyze user intent, messaging patterns, and engagement levels, QDate provides high-quality, intentional connections rather than high-volume, shallow interactions. This approach solves the common frustrations of choice overload for women and low signal-to-noise ratios for men, transforming the dating experience from a game of stimulation into an experiment in adaptive matching and constrained decision-making.

1. Introduction

Finding a meaningful connection in the digital age is challenging. Current dating apps often lead to frustration due to an overwhelming abundance of choices, which actually reduces the perceived value of each match. This project builds QDate, a dating application that prioritizes quality over quantity. Unlike traditional platforms that encourage endless swiping, QDate enforces artificial scarcity by providing users with only one match at a time. This constraints-based approach encourages intentionality and reflection, turning the dating process into a more thoughtful and data-driven experience.

2. Problem Statement

Current dating platforms suffer from a critical imbalance and a "paradox of choice" that reduces the quality of outcomes for all users. Men often experience extremely low signal-to-noise ratios and empty interactions, leading to reduced motivation due to intense competition. Conversely, women face a massive overload of options and shallow signals, which often results in decision paralysis or no decision at all.

This abundance of choice creates an environment where the perceived value of an individual connection is diminished. Furthermore, existing apps focus on maximizing swipes rather than learning true user intent. This "intentionality gap" leads to a cycle of frustration and superficial engagement, highlighting the urgent need for a system that uses artificial scarcity to encourage deeper reflection and more meaningful decision-making.

3. Objectives

- **Enforce Artificial Scarcity:** Develop a matching engine that restricts users to exactly one match at a time to increase intention and reflection.
- **Behavioral Data Collection:** Build a system to observe and record passive behavioral data, such as messaging frequency, reply latency, and balance of initiative, rather than relying solely on stated preferences.
- **Two-Phase Matching Lifecycle:** Implement a structured 14-day learning phase followed by a curated weekly matching cycle with built-in cooldown penalties for skipping.

- **Predictive ML Architecture:** Create a roadmap for a machine learning model that evolves from heuristic-based matching to predictive ranking based on conversion likelihood and communication compatibility.
- **Minimalist UX/UI:** Design a calm, non-stimulating interface that emphasizes clear countdowns and progress indicators to reinforce the "slow dating" philosophy.

4. User Stories

1. As a user, I want to receive only one match at a time, so that I can focus my full attention on one person without being distracted by other options.
2. As a user, I want to see a clear countdown timer for my current match, so that I am encouraged to engage thoughtfully within the 24-hour window.
3. As a user, I want the app to provide a minimalist and calm interface, so that I feel less overwhelmed and more intentional during my dating experience.
4. As a user, I want to provide feedback after a match expires, so that the system can learn my preferences and improve future suggestions.
5. As a woman, I want the system to limit the number of daily matches, so that I can avoid choice overload and decision paralysis.
6. As a man, I want to interact in a high-intention environment, so that my efforts aren't lost in a sea of empty interactions and extreme competition.
7. As a user in Phase 2, I want to receive one highly curated match per week, so that I can invest more emotional energy into a high-quality connection.
8. As a developer, I want to track passive behavioral data like reply latency and messaging frequency, so that the matching algorithm can learn true user intent beyond stated preferences.
9. As a user, I want to be notified of a "cooldown penalty" if I skip a curated match, so that I am reminded of the value and intentionality required by the platform.
10. As a user, I want to see progress indicators during the 14-day training period, so that I understand how the system is learning about my behavior.

5. Sample Test Code for User Stories

```
# User Story 1: Constrained Daily Match
# User Story 2: Behavioral Data Tracking (Passive Observation)
# User Story 3: Weekly Curated Match (Phase 2)
# User Story 4: Post-Match Feedback & Learning
```

6. Application Screenshots

Screenshot 1: Login & Intent Onboarding

The login screen reflects QDate's minimalist philosophy. Instead of high-stimulation visuals, it uses a calm aesthetic to set the tone for intentional dating.

- **Secure Access:** Standard login via email and password (with Google/Apple sign-in surfaced for future OAuth).
- **Initial Intent Setup:** A brief series of questions to establish the user's baseline preferences before entering the Phase 1 training period.

Screenshot 2: The Daily Focus (Phase 1)

This is the primary interface during the first 14 days, where choice is strictly constrained to increase reflection.

- **Single Profile View:** Only one potential match is displayed, emphasizing depth over breadth.
- **Scarcity Indicators:** A prominent 24-hour countdown clock encourages active engagement and timely decision-making.
- **Behavioral Progress:** A small indicator showing how many days remain in the "Learning Period" before graduating to curated matches.

Screenshot 3: Insights & Reflection Dashboard

Replacing traditional "swipe history," this dashboard shows the user how the system is learning their intent.

- **Compatibility Metrics:** Visual graphs showing communication compatibility and messaging patterns observed by the ML model.
- **Reflection Moments:** Prompts asking for lightweight feedback on expired matches to refine the predictive ranking model.
- **Intent Score:** A summary of the user's "intentionality level," rewarding high-quality interactions rather than match volume.

7. Typical User Flows

1. User Flow for Engagement During Phase 1 (Learning Period)

- » User is notified that their match of the day is ready
- » Opens the QDate app
- » Logs in securely
- » Views the single profile of the day, emphasizing scarcity over stimulation
- » Initiates a conversation within the 24-hour time-boxed window
- » System passively observes messaging frequency and reply latency
- » 24-hour window expires and the match is removed
- » User provides lightweight feedback on the interaction to help the system learn intent

2. User Flow for Phase 2 (Curated Weekly Match)

- » User is notified of their highly curated weekly match
- » Opens the QDate app
- » Navigates to the "Weekly Focus"
- » Reviews a deeply curated profile based on behavioral data from Phase 1
- » Decides to "Accept" (starting a high-investment interaction) or "Skip"
- » If "Skip" is chosen, the user sees a notice about the cooldown penalty to reinforce intentionality
- » User views their Intent Dashboard to see progress indicators and compatibility insights learned by the ML model

8. Implementation

8.1 Technologies Used (As Built)

The implemented system keeps the original product thesis intact while using a leaner, JavaScript/TypeScript-centric stack. The notable deviations from the initial proposal are called out inline.

Frontend (Client Application)

- **React Native (TypeScript) via Expo (SDK 54)** — Cross-platform mobile development (iOS & Android), delivered through the Expo Go runtime.

- **React Navigation (native-stack + bottom tabs)** — Dual-stream navigation that separates the "Daily Focus" (the real one-at-a-time match) from "Discover" (calibration / taste training) so both run independently.

- **Custom StyleSheet design system** — A calm, low-stimulation cream-and-sage theme implemented as lightweight design tokens (in place of the proposed Lottie/Tailwind) to reduce decision fatigue.

- **AsyncStorage** — Persists the auth session, the current phase, and a reload-surviving 24-hour / 7-day countdown timer (in place of SQLite).

- **Expo Image Picker & Image Manipulator** — Photo capture with client-side downscaling and compression for higher-quality profile images.

Behavioral Analysis & Processing

- **Server-side interaction telemetry** — Captures reply latency, message frequency, read latency, and time-of-day activity buckets, which feed the ML feature vectors.

- **Swipe-confidence signal** — Calibration swipes carry a response-time-based confidence weight used when learning a user's taste.

- **Intent scoring** — A 0–10 "intentionality" score adjusted on every match, skip, and connect, rewarding thoughtful engagement over volume.

- **Cooldown penalties** — Fixed cooling-off periods (24 hours in Phase 1, 7 days in Phase 2) enforce intentionality when a curated match is skipped (in place of the proposed rejection-velocity monitor).

Cloud Backend

- **Node.js + Express (TypeScript)** — REST API (run via tsx) serving authentication, daily/weekly matches, calibration decks, insights, polled real-time conversations, and moderation.

- **Custom authentication (bcrypt)** — Email + password with salted bcrypt hashing; Google/Apple are offered as sign-in options for a future OAuth integration (in place of Firebase Authentication).

- **In-process ML services (TypeScript)** — running inside the API rather than a separate Python service:

- Ranking engine: a weighted, learned-from-outcomes ranker blending intent, communication style, age, shared interests, appearance, activity overlap, cadence, engagement, and outcome confidence.

- Face-embedding pipeline: a pretrained face-recognition network (@vladmandic/face-api on TensorFlow.js) produces a 128-dimensional descriptor per user, taken from their clearest solo photo.

- Visual-taste model: a confidence- and recency-weighted Rocchio direction over face embeddings, scored mutually (geometric mean) so both people's attraction counts.

- **Reciprocity-aware matching** — a candidate is preferred only when the requester is also a near-top option for them, moving the assignment toward two-sided (stable) matches.

Database & Intelligence (Cloud)

- **MongoDB (Mongoose ODM) on MongoDB Atlas** — stores users, matches, swipes, messages, conversations, feedback, and reports (in place of PostgreSQL + pgvector):

- Taste vectors: interest and appearance preference vectors plus a 128-d face-taste vector are stored directly on the user document.

- Vector similarity: cosine similarity is computed in-process (application layer) to score interest/appearance alignment and face-taste compatibility.

- **Real-profile calibration** — the "Discover" decks train taste on real, gender- and age-eligible profiles: an Interests deck (bio + interest tags, no photo) and a Looks deck (face-recognized photos

only), replacing the originally planned static "Scale Profiles" library.

- **Trust & safety** — community-guidelines signing, in-app user reporting, and backend moderation that evaluates reports against the rules (warn / suspend / ban), plus optional biometric-consent gating before any face processing.